

CS437: Internet of Things

Lab1 Part 2

Name: Mohammad Tamim, Jacob Fuehne, Jeffery Fan, KyoSook Shin

NetIDs (include NetID of all group members): tamim2, jfuehne2, jfan16, kyosook2

Late days used: No

Video Link: <https://drive.google.com/file/d/1aMLnU5Xr9GChSfmn3XUME-alOz2j5T7m/view?usp=sharing>

Advanced Mapping:

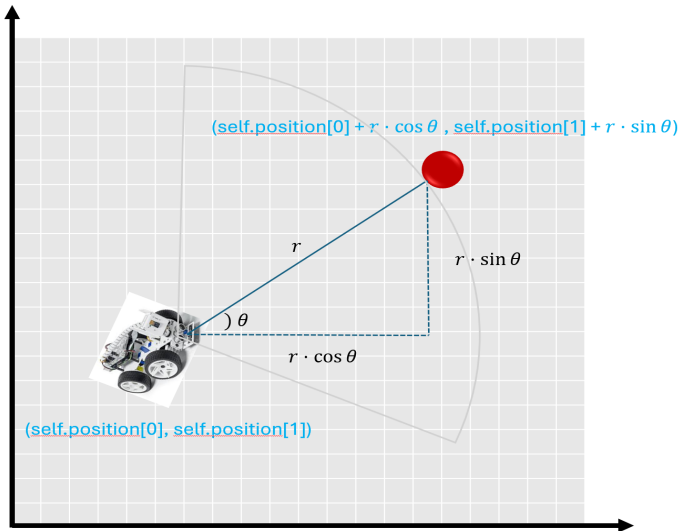
The foundation of our autonomous navigation system rests on creating a detailed and accurate representation of the environment. We developed a mapping system that maintains a globally available grid matrix to track obstacle locations in real-time. Through careful development and testing, we implemented a 150x150 grid system that provides the precise environmental awareness necessary for safe navigation.

Our mapping methodology centers on systematic environmental scanning. The PiCar-X performs wide-angle scans from -80 to +80 degrees using its ultrasonic sensor, measuring distances by analyzing sound wave reflections. The core technical specifications of our implementation include:

- Grid Resolution: 150x150 cells with 2cm per cell spacing
- Scanning Range: -80 to +80 degrees in 10-degree increments
- Position Tracking: Dynamic updates during movement
- Update Frequency: Continuous mapping with strategic rescans

To transform raw sensor measurements into usable spatial data, we developed a coordinate calculation system based on trigonometric principles. The system converts ultrasonic sensor readings into precise grid coordinates using these formulas:

$x\text{-coordinate} = \text{self.position}[0] + r \cdot \cos(\theta)$ $y\text{-coordinate} = \text{self.position}[1] + r \cdot \sin(\theta)$



These calculations enable us to convert the sensor's distance readings (r) and servo angle (θ) into exact positions within our coordinate system. The implementation balances precision with computational efficiency, using 2-centimeter grid cells to create detailed environmental representations while maintaining manageable processing requirements.

For real-time visualization and debugging, we created a `print_map()` function that generates an intuitive color-coded display of the environment. This visualization system uses:

- Red '1' markers to indicate detected obstacles
- Green '0' markers for clear, navigable paths
- Dynamic updates that reflect real-time environmental changes

```

Planning route to goal (85, 15)...
--- Starting Wide Area Scan ---

Current Environment Map (Red=obstacle, Green=clear):
0000000000000000
0000000000000000
0000010000000000
0000100000000000
0011000000000000
0010001000000000
0000010000000000
0000100000000000
000100000010100
000000000100100
000000000100100
000000000100100
00000000001000
0000000000000000
0000000000000000

Navigating to position (89, 11)
time elapsed: 0.18491888046264648
Path clear, moving ahead...

```

Through extensive testing and refinement, we addressed several critical challenges in the mapping system. Initial issues with heading accuracy during turns led us to implement Ackermann steering geometry principles, significantly improving our position tracking during

navigation. We enhanced our boundary detection algorithms to better handle obstacles at the scanning range edges, and implemented continuous mapping updates to maintain current environmental data during movement.

The system has demonstrated reliable performance across various test scenarios, successfully handling:

- Complex environments with multiple obstacles
- Dynamic obstacle arrangements
- Tight navigation spaces
- Real-time path adjustments

This mapping implementation provides the detailed environmental understanding needed for sophisticated path planning while maintaining real-time performance. We focused on creating a system that not only delivers accurate environmental data but also integrates smoothly with our navigation and path-planning algorithms. The result is a robust foundation for autonomous operation that effectively handles real-world navigation challenges.

The effectiveness of our approach is particularly evident in testing scenarios. The system consistently maintains accurate positioning information while providing reliable obstacle detection and representation. This reliability forms the basis for our path planning and navigation algorithms, enabling intelligent route selection and obstacle avoidance in dynamic environments.

Object Detection:

Our object detection system achieves robust real-time perception through careful optimization and strategic implementation choices. Working within the constraints of the Raspberry Pi platform, we developed a sophisticated approach using MediaPipe and TensorFlow Lite that balances performance requirements with detection accuracy.

The technical implementation builds upon these core specifications:

- Platform: Raspberry Pi 4 (8GB RAM)
- Detection Framework: MediaPipe with TensorFlow Lite
- Performance: 5.1-5.4 Frames Per Second
- Response Time: 200 milliseconds (exceeding human reaction speed of 250ms)
- Memory Usage: Approximately 1.7 GB

Hardware acceleration played a crucial role in achieving our performance targets. MediaPipe's built-in acceleration capabilities efficiently distribute processing loads across multiple CPU cores. Our system monitoring revealed consistent performance patterns:

- Base CPU utilization: 20-40% across cores
- Peak utilization: Occasional spikes to 80% during intensive processing
- Even distribution: Processing load balanced across all four cores
- Stable memory management: Consistent 1.7 GB usage maintaining system stability

A critical decision in our implementation involved rejecting multithreading approaches in favor of multiprocessing. This choice stemmed from detailed analysis of Python's Global Interpreter Lock (GIL) behavior with CPU-bound tasks. Traditional multithreading would actually hurt performance in our case, as image processing requires intensive CPU computation rather than I/O operations. MediaPipe's existing multiprocessing implementation already provides efficient parallel processing, making additional threading unnecessary and potentially counterproductive.

Our optimization efforts focused on several key areas:

Performance Optimization:

- Minimized unnecessary computational overhead
- Implemented efficient frame processing
- Balanced detection accuracy with processing speed
- Utilized hardware acceleration effectively
- Maintained consistent frame rates

Detection Strategy:

- Prioritized accurate single-frame object identification
- Focused on critical object detection (stop signs)
- Implemented confidence-based threshold management
- Maintained reliability across varying conditions

Through extensive testing and optimization attempts, we explored various performance improvements:

1. Temperature Management:
 - Tested CPU cooling solutions as suggested
 - Found no significant thermal bottlenecks affecting performance
 - Removed unnecessary cooling additions
2. Memory Optimization:
 - Verified automatic GPU memory allocation
 - Confirmed efficient memory usage patterns
 - Maintained stable performance without memory constraints
3. Processing Distribution:
 - Leveraged existing multiprocessing capabilities
 - Optimized core utilization patterns
 - Maintained balanced resource usage

The result is a system that demonstrates reliable performance across varying environmental conditions, successfully identifying and responding to relevant objects while maintaining consistent processing speeds. Our implementation achieves the critical balance between detection accuracy and computational efficiency necessary for autonomous navigation.

Looking ahead, we've identified potential areas for future optimization:

- Low-level library improvements
- External API processing options
- Enhanced detection algorithms

- Advanced hardware acceleration techniques

However, our current implementation successfully meets the project requirements while maintaining reliable performance within our hardware constraints. The system consistently identifies critical objects like stop signs, maintains appropriate response times, and integrates smoothly with our overall navigation system.

Key Performance Characteristics:

- Reliable object detection and classification
- Response times faster than human reaction
- Consistent frame rates in varying conditions
- Efficient resource utilization
- Stable memory management

Our object detection system represents a carefully engineered solution that balances technical constraints with practical requirements. By focusing on essential functionality and making intelligent implementation choices, we've created a system that reliably supports autonomous navigation while maintaining efficient resource utilization.



Self-Driving Navigation (Routing):

The Art of Smart Navigation: How Our Self-Driving Car Finds Its Way

Self-driving technology is about more than just moving forward it's about understanding the world, reacting to changes, and making intelligent decisions in real-time. Our self-driving system achieves this by combining A* pathfinding, live sensor data, and real-time object detection to navigate safely and efficiently.

But how does it actually work? Let's break it down.

Thinking Like a Navigator: The A* Algorithm

Finding the best path in an unknown environment is like solving a maze. Instead of taking random turns, our system uses the A* pathfinding algorithm, an intelligent approach that calculates the most efficient way to reach a destination while avoiding obstacles.

How A* Makes Smart Decisions

Unlike basic navigation methods that follow the shortest straight-line path, A* is smarter. It evaluates two key factors for each possible route:

1. g-score – The actual cost of moving from the start to the current point.
2. h-score – A smart estimate of how much further the goal is.

By combining these values ($f\text{-score} = g\text{-score} + h\text{-score}$), A* ensures that the car always chooses the most efficient and obstacle-free path. Instead of blindly following a direction, it dynamically recalculates its route based on real-time data.

From Theory to Reality: Implementing A* in Our Car

Mathematical models like A* are powerful, but bringing them to life in a real-world robotic system requires additional engineering. Our implementation does more than just calculate a path once it continuously updates its understanding of the environment.

- The system constantly scans for obstacles and updates its internal map.
- If a new obstacle appears, it recalculates a new path in real time.
- The car doesn't just move it actively thinks ahead and adapts to its surroundings.

This ensures that the car isn't just following a pre-planned route but is making intelligent navigation decisions at every moment.

Bringing Navigation to Life: Testing in a Controlled Environment

To ensure our system performs reliably, we designed a controlled testing area measuring 4.5 x 6 feet. This environment allows us to test the car's ability to handle realistic obstacles and complex routes.



- Starting Point (S) in bottom-right corner
- Goal Point (G) in top-left corner
- Boundary walls
- Potential obstacle placements)

Test Setup:

- Starting Point (S): Bottom-right corner.
- Goal Point (G): Top-left corner.
- Obstacles: Placed to test navigation around unexpected barriers.
- Boundary Walls: Six-inch-high barriers creating a defined space.

This setup allows us to analyze and improve the car's ability to navigate efficiently while responding to real-world challenges.

The Two Pillars of Smart Navigation

Our self-driving system relies on two fundamental components that work together to create smooth and intelligent movement.

1. Environmental Awareness: Seeing the World in Real Time

The car's ability to navigate depends on its understanding of the environment. To achieve this, it uses:

- A 360-degree scanning system that constantly collects data.
- A ± 80 -degree field of view for detecting obstacles.
- 10-degree incremental measurements for high precision.
- Noise filtering algorithms to improve sensor accuracy.

By constantly monitoring and updating its surroundings, the car can avoid unexpected obstacles and make quick adjustments when needed.

2. Dynamic Movement Control: Precision in Motion

Navigation isn't just about choosing the right path it's also about moving accurately and efficiently. Our system fine-tunes movement through:

- Forward Speed: 25 units.
- Turning Speed: 15 units.
- Backup Speed: 8 units.
- Servo Offset Correction: -6.1° to ensure straight-line movement.

With precise control over speed and direction, the car can handle tight turns, navigate confined spaces, and avoid unnecessary stops.

Adapting to the Unexpected: Real-Time Obstacle Avoidance

Autonomous cars must react instantly to changes in their environment. To achieve this, we designed a three-tier safety system that helps the car make quick decisions when facing obstacles.

How the Car Responds to Different Zones:

- Safe Zone (above 43 cm): The car continues moving normally.
- Warning Zone (30-43 cm): The system increases scanning frequency and prepares to adjust its route.
- Danger Zone (below 19 cm): The car stops immediately and executes an avoidance maneuver.

By constantly updating its obstacle map, the car can make split-second decisions that keep it on track without unnecessary stops or collisions.

What Makes Our Navigation Unique?

Many navigation systems calculate a path once and blindly follow it even if new obstacles appear. Our system improves upon this by integrating real-time mapping and continuous route optimization.

Key Innovations:

- Periodic Rescanning: After every turn or backup, the car updates its map to reflect changes in the environment.
- Live Path Recalculation: If a new obstacle appears, the system immediately adjusts the route instead of following an outdated plan.
- Adaptive Decision-Making: The car doesn't just follow a pre-set routine it analyzes multiple possibilities before making a move.

This flexible approach allows the system to handle complex environments with a high degree of intelligence.

Challenges and How We Solved Them

Developing an intelligent self-driving system comes with its fair share of obstacles. Some of the biggest challenges we faced and how we fixed them include:

Challenge: Inaccurate Turns Solution: Fine-tuned servo offsets to improve steering precision.

Challenge: Delayed Obstacle Detection Solution: Used asynchronous processing to reduce lag between detection and action.

Challenge: Unnecessary Stops Solution: Optimized scanning frequency to prevent interruptions in motion.

Each of these refinements made our system faster, smarter, and more efficient in real-world conditions.

The Journey Ahead: Making Machines Smarter

Our self-driving system is more than just a car it's a learning machine that makes intelligent decisions in real time. By integrating:

- A* pathfinding for smart route selection.
- Live scanning for real-time environmental updates.
- Adaptive movement strategies for handling obstacles.

We have built a system that doesn't just follow instructions it thinks, learns, and reacts like a real driver.

Through continuous refinement and testing, we aim to make this navigation system even smarter, faster, and more adaptable to real-world challenges.

This project is a step forward in making truly intelligent autonomous navigation a reality.

Full Self-Driving:

The culmination of our project brings together advanced mapping, object detection, and navigation systems into a cohesive autonomous platform. This integration transforms our PiCar-X into a sophisticated self-driving vehicle capable of understanding its environment, making intelligent decisions, and navigating safely to its goals. Through careful system architecture and thoughtful implementation, we've created a platform that demonstrates the potential of integrated robotics and IoT technologies.

Our system architecture creates a seamless flow of information between components. At its foundation lies the advanced mapping system, which maintains a dynamic 150x150 grid representation of the environment. This mapping component continuously updates through servo-mounted ultrasonic sensor readings, scanning from -80 to +80 degrees in precise 10-degree increments. The scanning process generates a detailed environmental model where each cell represents a 2cm square of real space, providing the granular detail necessary for precise navigation.

Real-time object detection operates alongside our mapping system, leveraging MediaPipe and TensorFlow Lite to process camera inputs at 5.1-5.4 frames per second. This vision system achieves response times of approximately 200 milliseconds – faster than human reaction speed – while maintaining memory usage at a stable 1.7GB. Through efficient multiprocessing implementation, we distribute the computational load across CPU cores, typically maintaining 20-40% utilization with occasional peaks to 80% during intensive processing.

The integration of these systems enables sophisticated decision-making capabilities. When the car encounters a stop sign, the object detection system immediately identifies it, triggering a predefined response protocol. The navigation system then brings the vehicle to a complete stop, maintaining this state for the required duration before proceeding. Similar protocols exist for other detected objects, with each triggering appropriate responses based on the object type and context.

Our navigation logic operates through a multi-stage process that ensures safe and efficient movement:

The first stage involves environmental scanning and mapping. The ultrasonic sensor performs wide-angle scans, creating an initial grid representation of surroundings. This map undergoes continuous updates as the vehicle moves, ensuring our navigation decisions always reflect current conditions. Crucially, our implementation runs these updates asynchronously, eliminating potentially dangerous delays in obstacle detection and response.

The second stage employs A* pathfinding to determine optimal routes. Working with our grid-based map, the algorithm considers both distance and obstacle placement to plan efficient paths to the goal. The system regularly recalculates these routes based on new environmental data, allowing for dynamic adaptation to changing conditions. We enhanced this process by implementing Ackermann steering geometry calculations, which provide precise control over turning movements and improve our position tracking accuracy.

Movement execution forms the third stage, where our system translates planned routes into precise control commands. We carefully calibrated speed settings for different operations: 25 units for forward movement, 15 units for turns, and 8 units for backup maneuvers. A -6.1° servo offset compensation ensures straight-line movement accuracy. The system continuously monitors its surroundings through three safety zones: a safe zone beyond 43cm, a warning zone between 30-43cm, and a danger zone within 19cm, each triggering appropriate responses to maintain safe operation.

Performance testing revealed the effectiveness of our integrated approach. The system consistently achieves its navigation goals while maintaining safety parameters and responding appropriately to detected objects. Our implementation demonstrates particular strength in handling complex scenarios requiring multiple competencies, such as navigating around obstacles while maintaining awareness of traffic signals and adjusting routes based on real-time environmental changes.

Safety considerations remain paramount throughout our integration. The system maintains conservative movement strategies, ensuring complete stops for critical detected objects and maintaining safe distances from potential obstacles. Our confidence-based detection threshold management helps prevent false positives while ensuring reliable identification of important objects like stop signs.

The computational approach balances sophisticated functionality with practical resource constraints. Through careful optimization and thoughtful implementation choices, we maintain responsive performance while handling complex environmental interactions. The system demonstrates consistent reliability across varying conditions, adapting to different lighting conditions, obstacle configurations, and navigation challenges.

Looking ahead, we've identified several promising areas for enhancement. Machine learning-based predictive obstacle detection could improve response times, while more advanced probabilistic mapping techniques might enhance environmental understanding. The modular nature of our implementation allows for straightforward integration of such improvements, enabling continuous refinement of system capabilities.

Through this integration, we've created a platform that demonstrates the potential of autonomous navigation systems. Our implementation balances sophisticated technical capabilities with practical reliability, creating a robust foundation for autonomous operation. The system showcases how carefully integrated components can work together to create an intelligent, responsive platform capable of safe and effective autonomous navigation.

Name	Contribution
Mohammad Tamim	• Enhanced mapping with color-coded visualization (red for obstacles, white for clear space) • Developed comprehensive navigation and obstacle avoidance system • Implemented A* pathfinding for optimal route planning • Added dynamic obstacle detection and avoidance capabilities • Integrated advanced path planning with base mapping • Implemented continuous mapping updates during navigation • Enhanced heading calculations using Ackermann steering geometry
KyoSook Shin	• Developed initial mapping framework and grid system • Created base grid representation system with 150x150 resolution • Implemented basic movement and scanning functions • Provided foundation for ultrasonic sensor integration • Set up initial environment scanning capabilities • Established core scanning algorithms and distance calculations • Created initial visualization system for debugging
Jacob Fuehne	• Implemented object detection using MediaPipe and TensorFlow Lite • Achieved consistent 5.1-5.4 FPS performance with efficient CPU utilization

	• Optimized multiprocessing implementation for improved responsiveness • Added async functionality for better system response times • Enhanced system reliability through extensive testing and debugging • Documented performance characteristics and optimization strategies • Integrated camera-based recognition with navigation systems
Jeffery Fan	• Managed final system integration across all components • Led comprehensive testing and documentation efforts • Enhanced boundary visualization and detection • Optimized final implementation for improved efficiency • Validated system requirements and performance metrics • Supported component integration and testing • Improved obstacle representation and mapping accuracy